Attestation & assertion workflow

Step-by-step, with explicit ownership at every step. Companion to `architecture-decisions.md` (product/protocol) and `appattestkit-module-design.md` (module internals).

Ownership rule: the module owns anything touching Apple's rate-limited resources; the app owns anything touching your protocol.

Part 1 — Registration (attestation)

Runs **once per install, ever**. Silent — no user input at any point.

Overview

#	Step	Owner	Touches
1	Generate identity keypair	**App**	Keychain
2	Check `isSupported`	Module	DeviceCheck
3	Load or generate App Attest key	**Module**	Secure Enclave ⚠️ rate-limited
4	Persist `keyId`	**Module**	Keychain
5	Fetch challenge	App → Module calls it	Network
6	Build `clientDataHash`	**App**	—
7	Call `attestKey`	**Module**	Apple ⚠️ one-shot
8	Persist attestation object	**Module**	File (protected)
9	Submit to server	**App**	Network
10	Server verifies	Server	—

#	Step	Owner	Touches
11	Store account UUID	**App**	Keychain / UserDefaults
12	Clear cached attestation	**Module**	File

Step 1 — Identity keypair (App)

let identityKey = Curve25519.KeyAgreement.PrivateKey()

try keychain.store(identityKey.rawRepresentation, for: .identityKey)

X25519, stored in **Keychain** with `kSecAttrAccessibleWhenUnlockedThisDeviceOnly`. Cannot live in the Secure Enclave — it only supports P-256 (module doc §2a).

Generate this **before** attestation: step 6 needs the public half, and generating it first means a failed attestation doesn't leave you re-deriving it.

> **App-owned** because it is a protocol concern. The module's dependency constraint is `DeviceCheck` + `CryptoKit` + `Foundation`; it must not know your handshake exists.

Step 2 — Support check (Module)

`DCAppAttestService.shared.isSupported` is false in the simulator, on jailbroken devices, and under some enterprise configurations. Terminal state — decide the app's policy explicitly (architecture doc §8).

Step 3 — Load or generate App Attest key (Module) ⚠️

if let existing = try store.loadKeyId() { return existing }

let keyId = try await service.generateKey()

Load before generate, always. Apple caps key generations per device over its lifetime. A retry loop that regenerates on failure will permanently lock the user out.

> **Module-owned.** If the app held the `keyId`, it could regenerate at will and the "never regenerate except on `invalidKey`" rule would stop being enforceable.

Step 4 — Persist `keyId` (Module)

Written to Keychain **before** step 7. A crash between generate and attest must be recoverable, or the key is orphaned and a generation is wasted.

State → `.keyGenerated(keyId:)`

Step 5 — Fetch challenge (App implements, module calls)

```
let challenge = try await transport.fetchChallenge()   // 32 bytes
```

Server-side TTL **~15 minutes** for registration — longer than the 60s session TTL, because a cached attestation is bound to this challenge and the key can only be attested once (module doc §7a).

> Module calls it; app implements `AttestationTransport`. The module never knows a URL.

Step 6 — Build `clientDataHash` (App)

```
let clientDataHash = Data(SHA256.hash(data: challenge +
identityKey.publicKey.rawRepresentation))
```

This is the binding that makes the whole scheme work. The App Attest key signs over a hash containing the identity public key, letting the server conclude that this identity key came from a genuine app on real hardware. Without it, any identity key could be attached to any valid attestation.

> **App-owned** — the composition is app-specific. The module receives an opaque hash.

Step 7 — Attest (Module) ⚠️

```
let attestation = try await service.attestKey(keyId, clientDataHash: clientDataHash)
```

One-shot. After success the key becomes assertion-only. A later retry may fail with `invalidKey`, and the client **cannot distinguish** "never attested" from "attested, receipt lost" — which is precisely why step 8 exists.

Returns ~5KB of CBOR containing the certificate chain.

Step 8 — Persist attestation (Module)

Written **immediately on return, before any network call.**

Stored as a file with `NSFileProtectionComplete` in Application Support — 5KB is large for Keychain, and it is a public credential, not a secret.

State → `.attestationPending(keyId:attestation:challenge:)`

Resuming from this state skips Apple entirely and retries only the upload.

Step 9 — Submit (App implements)

POST /register

{ keyId, challenge, attestation, identityPublicKey }

All four fields are **public** credentials. TLS in transit is sufficient; no application-layer encryption needed. Add certificate pinning (module doc §2a).

Step 10 — Server verification (Server)

In order, all mandatory:

1. Challenge exists, unconsumed, unexpired → mark consumed
2. Parse CBOR; validate `x5c` chain to Apple's App Attest root
3. Extract nonce from leaf cert extension `1.2.840.113635.100.8.2`
4. `nonce == SHA256(challenge ‖ identityPublicKey)`
5. `authData.rpIdHash == SHA256(teamId + "." + bundleId)`
6. Counter is 0
7. `keyId == SHA256(attestedPublicKey)`

Stores `account_uuid`, `key_id`, `attest_pubkey`, `counter = 0`.

Discards `identityPublicKey` after step 4 — it is the most graph-linkable value on the server and has no further use (architecture doc §3, flow 1).

Idempotent on `keyId`: if already registered, return the existing UUID rather than erroring. Handles the lost-response case.

Step 11 — Store account UUID (App)

App-owned: it is your account model, not the module's.

Step 12 — Clear cached attestation (Module)

Only **after** server confirmation. State → `.attested(keyId:)`

Part 2 — Assertion

Runs on **every authenticated request**, forever. Deliberately thin: no state machine, no retries, no persistence.

Overview

#	Step	Owner
1	Decide a request needs auth	**App**
2	Obtain a nonce	**App**
3	Build payload hash	**App**
4	Sign	**Module** (`sign()`)
5	Send	**App**
6	Verify + counter check	Server

Steps 1–3 (App)

```
let body = try JSONEncoder().encode(request)

let nonce = try await api.fetchSessionNonce()          // 60s TTL

let payloadHash = Data(SHA256.hash(data: nonce + body))
```

Step 4 — Sign (Module)

```
let assertion = try await coordinator.sign(payloadHash)
```

Throws `.notAttested` unless state is `.attested`.

> **Module performs, app drives.** Signing needs the `keyId`, which the app never sees. But *when* and *what* to sign is entirely the app's decision.

Keep this path thin — no I/O, no retries, no observer calls. All the machinery in the module doc belongs to the one-time flow.

Step 5 — Send (App)

POST /session

{ keyId?, assertion, nonce, body }

The server needs to locate the stored public key. Whether that is via `keyId` in the body or a session identifier is a server contract decision — see open questions.

Step 6 — Server verification

1. Nonce valid and unconsumed
2. Recompute `SHA256(nonce + body)`, compare to the assertion's `clientDataHash`
3. Verify the ECDSA signature against the stored `attest_pubkey`
4. **Counter strictly greater than stored**, then persist the new value
5. Issue a short-lived session token

The counter is server-side state. Apple increments it inside each assertion; the client never tracks it. Do not add counter handling to the module.

Amortise: verify once at WebSocket handshake, then use the session token. Do not assert per message.

Part 3 — Re-attestation ⚠️ BLOCKED

Triggered by `DCError.invalidKey` — Keychain cleared, device restored from backup, or key otherwise invalidated.

Client side (implementable now):

1. Discard the stored `keyId`, clear cached attestation
2. Generate a new App Attest key
3. Re-attest, binding the **same** identity public key
4. Prove possession by signing the challenge with the identity private key
5. Submit as a re-attestation rather than a fresh registration

Server side (blocked): `/register` is idempotent on `keyId`. A returning user arrives with a *new* `keyId` and the *same* identity key — currently that produces a duplicate account or a rejection.

Needs: accept a new `keyId` bound to an existing identity key, gated on proof of possession of the identity private key.

Open sub-questions (module doc §8):

- Does the identity key survive the events that invalidate an App Attest key? Both are in Keychain, so probably — but device restore needs verifying.
- If the identity key is *also* lost, the user is new and must re-pair. Acceptable? How is it surfaced?
- Rate-limit re-attestation separately; it is otherwise an account-takeover surface.

Failure paths quick reference

Failure	**Recovery**	**Owner**
No network at launch	Let the user in, mark unregistered, retry on foreground + connectivity change	App drives, module resumes
`isSupported == false`	Terminal; app policy decides	App
`attestKey` transient failure	Retry **same `keyId`**, exponential backoff	Module
Crash after `attestKey`	Resume from `.attestationPending`, upload only	Module
Crash after upload, before response	Server idempotency returns existing UUID	Server
Cached attestation stale (challenge expired)	Regenerate key — **capped at 2–3 ever, persisted**	Module
`DCError.invalidKey`	Re-attestation flow — **blocked on server contract**	Both
Keychain survives app deletion	`UserDefaults` flag absent → purge Keychain on first launch	App

The cardinal rule: never call `generateKey()` on failure except for `invalidKey` and the capped stale-challenge case. Everything else retries the existing `keyId`.